THE MAGIC EXPRESS

Speeding Up Lists &

Python Programming | Grade 4

PLANRS Academy

Imagine This...

Imagine you have a giant basket of mangoes. Some are ripe and yellow (Alphonso!), and some are still green.

Visual: A child picking mangoes one by one vs. using a magic sieve that filters them instantly.

If you pick them one by one, it takes forever! But what if you had a **Magic Sieve**? You pour the basket in, and *BAM!*—only the ripe ones fall into your tray.

In Python, **List Comprehension** is our Magic Sieve. It’s the "Express Route" to making lists faster and smarter!

One Line Instead of Many

Usually, we use a 'for-loop' to make a list. That takes many lines. With List Comprehension, we do it in just **ONE** line!

The Blueprint:

[x for x in list]

It’s like taking the **Vande Bharat Express** instead of a slow passenger train. You get the same result, but much faster!

Visual: Comparison of a long winding road (Loop) and a straight high-speed train track (Comprehension).

Old Way vs. New Way

Let's look at how Reyansh creates a list of numbers:

OLD WAY (THE LONG ROAD)

```
numbers = [1, 2, 3, 4]
new_list = []
for n in numbers:
  new_list.append(n * 2)
# Takes 4 lines!
```

NEW WAY (THE MAGIC EXPRESS)

```
numbers = [1, 2, 3, 4]
new_list = [n * 2 for n in numbers]
# Just 1 line!
```

Both do the same thing, but the Magic Express is cleaner and easier to read.

Organizing Your World

Sorting means putting things in order. Think of a **Kirana Store** shopkeeper arranging spice packets from the smallest price to the largest.

Visual: A shopkeeper neatly arranging jars on a shelf.

Two ways to sort:

- **.sort()** — Changes your original list (Permanent).
- **sorted()** — Keeps the old list and gives you a new, tidy one!

```
prices = [50, 10, 30]
prices.sort()
print(prices) # Result: [10, 30, 50]
```

Activity: Gali Cricket Scoreboard

Advait and his friends played cricket. Here are their scores: `[12, 7, 19, 5, 22]`. Let's find out who the top scorer is!

Ascending (Low to High)

```
scores = [12, 7, 19, 5, 22]
print(sorted(scores))
# [5, 7, 12, 19, 22]
```

Descending (High to Low)

To go from top to bottom (like coming down temple stairs), we use `reverse=True`.

```
print(sorted(scores, reverse=True))
# [22, 19, 12, 7, 5]
```

The winner scored 22 runs!

Why Use This?

Coding Ninjas use these tools to keep their code organized, just like a **Librarian** arranges books or a student lines up for the **Morning Assembly**.

Quick Quiz:

1. Which is faster: A for-loop or List Comprehension?
2. If we climb temple stairs, is that **Ascending** or **Descending**?
3. How do you sort a list from High to Low?

Awesome job today! You are now a List Comprehension Ninja! 🥷✨